

MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES
(A Constituent unit of MAHE, Manipal)

Linux OS and Scripting – Process_Lab

Reg. Number	Name	Branch
261100690006	MOHAMMED HARIS	Cybersecurity

31/10/2026



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Exercise 1: Process identification
Display all running processes with their PID, PPID, owner, CPU usage, memory usage, and command.

```
ps -eo pid,ppid,user,%cpu,%mem,cmd
```

List only the processes belonging to the current user.

```
ps -u $USER -o pid,ppid,user,%cpu,%mem,cmd
```

Identify the five processes consuming the most CPU.

```
ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%cpu | head -6
```

Identify the five processes consuming the most memory.

```
ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%mem | head -6
```

Explain why high CPU usage alone does not confirm malicious activity.

High CPU usage is normal for many legitimate tasks, such as compiling, video encoding, antivirus scans, and software updates. To judge whether a process is malicious we also need its owner, parent process, binary location, start time, network connections, and whether the name or path looks suspicious.

Exercise 2: Parent and child processes
Write a shell script that starts two background processes and displays their PIDs.

```
#!/bin/bash
echo "Script PID:  $$"
echo "Script PPID: $PPID"
sleep 300 &
echo "Child 1 PID: $!"
sleep 400 &
echo "Child 2 PID: $!"
echo "Children of this script:"
ps -o pid,ppid,cmd --ppid $$
wait
```

Identify the PID and PPID of the shell script.

```
chmod +x parent_child.sh
./parent_child.sh
```

Display the child processes created by the script.

```
ps -o pid,ppid,cmd --ppid <SCRIPT_PID>
```

Explain the purpose of $$, $!, &, and wait.
$$: PID of the current shell/script
$!: PID of the most recent background process
&: runs a command in the background
wait: pauses the script until its background children finish
Observe what happens to the child processes when the parent script terminates.

```
kill <SCRIPT_PID>
```

```
ps -eo pid,ppid,cmd | grep "sleep"
```

Exercise 3: Process control using kill
Start a background process and record its PID.

```
sleep 600 &
PID=$!; echo $PID
```

Suspend the process and verify its state.

```
kill -SIGSTOP $PID
ps -o pid,stat,cmd -p $PID
```

Resume the process and verify its state.

```
kill -SIGCONT $PID
ps -o pid,stat,cmd -p $PID
```

Terminate the process gracefully.

```
kill -SIGTERM $PID
```

Verify that the process has terminated.

```
ps -p $PID
```

Explain the differences between SIGSTOP, SIGCONT, SIGTERM, and SIGKILL.

SIGSTOP: Pauses the process; cannot be caught or ignored
SIGCONT: Resumes a stopped process
SIGTERM: Polite request to terminate; can be caught, allowing cleanup
SIGKILL: Immediate kill by the kernel; cannot be caught, no cleanup

Exercise 4: Graceful and forced termination
Write a monitoring script that periodically records a timestamp in a log file.
Configure the script to record a shutdown message when it receives SIGTERM.

```
#!/bin/bash
LOG=~/proc_lab/monitor.log
trap 'echo "$(date) - SIGTERM received, shutting down graceFully" >> "$LOG"; exit
0' TERM

echo "$(date)- started, PID $$" >> "$LOG"
while true; do
    echo "$(date) - heartbeat" >> "$LOG"
    sleep 5 &
    wait $!
done
```

Terminate the script using SIGTERM and inspect the log.

```
chmod +x monitor.sh
./monitor.sh &
```

```
MPID=$!
sleep 12
kill -TERM $MPID
cat monitor.log
```

Restart the script and terminate it using SIGKILL.

```
./monitor.sh &
MPID=$!
sleep 12
kill -9 $MPID
tail monitor.log
```

Compare the results and explain why they differ.
SIGTERM can be caught, so the trap handler runs and writes the shutdown message. SIGKILL cannot be caught or handled, so the kernel destroys the process instantly and no cleanup code runs.

Exercise 5: Multiple processes using killall
Start three instances of a lab program with a unique process name.

```
cp $(which sleep) ~/Documents/LoS/Lab/Process_Lab/labproc // doesn't work on latest
os

cat > "$LAB/labproc" << 'EOF'
#!/bin/bash
exec -a labproc sleep "${1:-900}"
EOF
chmod +x "$LAB/labproc"

~/process_lab/labproc 600 &
~/process_lab/labproc 600 &
~/process_lab/labproc 600 &
```

Display the PID and owner of each instance.

```
ps -C labproc -o pid,user,cmd
```

Terminate one instance using its PID.

```
kill <ONE_PID>
```

Terminate the remaining instances using killall.

```
killall labproc
```

Verify that all instances have terminated.

```
pgrep -a labproc
```

Explain the risk of using killall with a common process name.
It kills every process matching the name. With a common name like python, bash, java, or sshd, we could kill unrelated or critical processes, and as root even other users' processes.

Exercise 6: Recurring process monitoring using cron

Write a shell script that records the current date, time, username, and number of processes owned by the current user.

```
#!/bin/bash
echo "$(date '+%F %T') user=$(whoami) processes=$(ps -u $(whoami) --no-headers | wc -l)" >> /home/haris/process_lab/cron_monitor.logs
```

Schedule the script to run every minute.

```
chmod +x cron_monitor.sh
crontab -l > ~/process_lab/crontab.backup 2>/dev/null
crontab -e
```

```
* * * * * /home/haris/process_lab/cron_monitor.sh
```

Verify that the scheduled task executes successfully.

```
systemctl status cron
tail -f ~/process_lab/cron_monitor.log
```

Start additional background processes and observe changes in the log.

sleep 500 &

Modify the schedule to run every five minutes.

```
*/5 * * * * /home/haris/process_lab/cron_monitor.sh
```

Remove the exercise entry without deleting other cron entries.

```
crontab -l | grep -v "cron_monitor.sh" | crontab -
crontab -l
```

Exercise 7: Cron scheduling

Write cron entries for the following requirements:

Check failed-login logs every five minutes.

```
*/5 * * * *   /usr/local/bin/check_failed_logins.sh
```

Capture a process snapshot at the beginning of every hour.

```
0 * * * *     /usr/local/bin/process_snapshot.sh
```

Back up security logs daily at 2:30 AM.

```
30 2 * * *    /usr/local/bin/backup_security_logs.sh
```

Generate a security report every weekday at 9:00 AM.

```
0 9 * * 1-5   /usr/local/bin/security_report.sh
```

Perform a file-integrity check every Sunday at 3:00 AM.

```
0 3 * * 0     /usr/local/bin/integrity_check.sh
```

Run a monitoring script every 15 minutes.

```
*/15 * * * *  /usr/local/bin/monitor.sh
```

Generate a weekly report every Monday at 8:30 AM.

```
30 8 * * 1    /usr/local/bin/weekly_report.sh
```

Exercise 8: One-time scheduling using at
Schedule a task to record the current date and process list two minutes from now.

```
echo 'date >> ~/process_lab/at_output.txt; ps aux >> ~/process_lab/at_output.txt' |
at now + 2 minutes
```

Display all pending jobs.

```
atq
```

Inspect the commands associated with the scheduled job.

```
at -c <JOB_NUMBER>
```

Verify that the output file is created after execution.

```
ls -l ~/process_lab/at_output.txt
cat ~/process_lab/at_output.txt
atq
```

Explain why the completed job no longer appears in the pending-job list.

at jobs run once. After execution the job is removed from the queue, so only pending jobs are listed

Exercise 9: Cancelling scheduled jobs
Schedule a harmless task to execute ten minutes from now.

```
echo 'echo hello >> ~/process_lab/harmless.txt' | at now + 10 minutes
```

Identify its job number.

```
atq
```

Cancel the scheduled task.

```
atrm <JOB_NUMBER>
```

Verify that the task has been removed.

```
atq
```

Explain the difference between cancelling an at job and removing a cron entry.

An at job is one-time, identified by a job number, and cancelled with atrm. A cron entry is recurring, stored as a line in a crontab, and removed by editing the crontab and deleting that line. Until we do, it keeps running forever.

Exercise 10: Incident-response scenario
An unfamiliar lab process has multiple running instances and a scheduled task that will restart it.

```
#!/bin/bash
LAB="$HOME/Documents/LoS_Lab/process_lab"
```

```
mkdir -p "$LAB"
```

```
#!/bin/bash
exec -a labproc sleep "${1:-900}"



chmod +x "$LAB/labproc"

echo "$LAB/labproc 900 &" | at now + 10 minutes
```

Identify all instances of the lab process and record their PIDs and owners.

```
ps -eo pid,ppid,user,stat,cmd | grep [l]abproc
```

Suspend one instance and verify its state.

```
kill -SIGSTOP <PID>; ps -o pid,stat,cmd -p <PID>
```

Resume the suspended instance.

```
kill -SIGCONT <PID>; ps -o pid,stat,cmd -p <PID>
```

Terminate all instances of the lab process.

```
pkill -TERM labproc
```

Identify and inspect the scheduled restart task.

```
atq
at -c <JOB>
crontab -l
sudo ls /etc/cron.d /var/spool/cron
```

Cancel the scheduled restart.

```
atrm <JOB>
```

Verify that no instances or restart jobs remain.

```
pgrep -a labproc
atq
crontab -l
```